Computer Vision Challenge Round 1 Proposal

Vision System Design and Implementation

Proposal & Team Details

Proposal Title: Real-time 3-wheeler Adas for Unstructured Indian road conditions

Team Name: AllianceNexus

Team Lead: Kavya Giri

Email – goswamikavya626@gmail.com

Institution/Organisation: Alliance university

Challenge Track: AI and Computer Vision

Team Members, Affiliation & Expertise

Member 1 (Team Lead): Kavya Giri – Alliance university – System Architecture, Logic Fusion, and Project Integration.

Member 2 : Joydip banerjee– Alliance university – Computer Vision, YOLO11-Seg Implementation, and Semantic Segmentation.

Member 3 : Yuvraj Singh – Alliance university – Deep learning, Model optimization and monocular depth estimation.

Member 4 : Farhan Parwaz – Alliance university – Embedded Systems, Edge Deployment (OpenVINO/ONNX), and Real-time Pipeline Optimization.

Understanding of the Problem Statement

The problem : Standard ADAS systems are designed for structured Western roads and fail in the “unstructured” traffic conditions of India, characterized by unpredictable obstacles like cows, potholes, and aggressive maneuvering.

Key objective: To develop a real –time safety and collision-warning system optimized for the unique narrow-front/wide-back physics of a three-wheeled auto-rickshaw.

Type of visual task: A multi-task learning approach involving simultaneous object detection(vehicles/animals), segmentation (drivable area), and Monocular Depth Estimation.

Real – world impact – Improving road safety for millions of commuters by providing low-latency “warning or stop” alerts on affordable, edge-deployed hardware.

Proposed Solution – High Level Architecture

Overview of the **computer vision pipeline**

Typical pipeline:

Input Data: Asynchronous USB camera feed

→ Preprocessing: (Contrast Enhancement) and Resizing (256×256). CLAHE

→ Feature Extraction: mobileNetV2-based lightweight backbone.

→ Model Inference: parallel execution of YOLOv11-seg and MiDaS small.

→ Post Processing: Triangular ROI intersection check.

→ Output Prediction: HUD overlay (safe/warning/stop).

Include a **block diagram of the system architecture**

Methodology

Explain the **step-by-step workflow**

Dataset collection / usage: leveraging Indian driving dataset (IDD) for local road context.

Image preprocessing & augmentation: Simulating monsoon rain, dust, and high- beam glare.

Model training: Transfer learning from MS-COCO to Indian-specific classes.

Validation and hyperparameter tuning: Optimizing for recall to ensure zero-miss detection of obstacles.

Deployment or inference pipeline: using a python-based inference loop with threading to maintain high FPS .

Techniques & Algorithms

Describe the **core algorithms used**

- CNN architectures (ResNet / EfficientNet / YOLO / UNet): YOLO-11-Nano segmentation (for simultaneous detection and road masking).
- Feature extraction methods: Depthwise separable convolutions to reduce parameter count.
- Object detection / segmentation techniques: The system uses **YOLO11n-seg** for simultaneous object detection and drivable area segmentation , utilizing a **COCO-pretrained backbone** fine-tuned on the **India Driving Dataset (IDD)** to adapt to unstructured traffic.
- Transfer learning strategy: The system utilizes **YOLO11n-seg** for simultaneous object detection and drivable area segmentation , fine-tuning a **COCO-pretrained backbone** on the **India Driving Dataset (IDD)** to adapt to unstructured traffic and local vehicle classes.
- Model optimization methods: OpenVINO/TensorRT conversion for real-time CPU performance.

Dataset & Training Strategy

Describe:

- Dataset used (public or custom)
- Data preprocessing steps: Combination of India driving Dataset (IDD) and nuScenes.
- Data augmentation techniques : Random scaling , translation, and simulated lighting variations.
- Training-validation-test split: The dataset is divided into a **70% Training** set for model optimization, a **20% Validation** set for hyperparameter tuning, and a **10% Test** set for final unbiased performance evaluation.
- Hardware used for training (GPU / cloud): Trained on NVIDIA GPUs; developed on standard laptop (Intel/AMD CPU).

Real-Time Processing & Performance

Explain system performance considerations:

- Inference speed: Target of 15-30 FPS on a standard laptop.
- Latency handling: Asynchronous frame grabbing to prevent UI lag.
- Model size and efficiency: Under 15MB to ensure low memory footprint.
- Edge deployment capability: The system utilizes **INT8/FP16 Quantization** and **Multi-threaded Asynchronous Inference** via **OpenVINO or ONNX Runtime** to achieve **15–30 FPS** on standard CPU hardware with a sub-15MB memory footprint.

Also discuss robustness against:

- Noise : Employs **Kalman Filtering** to stabilize bounding boxes and masks against high-frequency vehicle vibrations.
- Lighting variation: Uses **CLAHE (Contrast Limited Adaptive Histogram Equalization)** to maintain detection accuracy under harsh sunlight and night-time glare.
- Occlusions: Optimized **Recall** in the YOLO11n-seg backbone ensures consistent tracking of partially hidden pedestrians or animals.

Evaluation Metrics

Define how performance will be measured.

• Accuracy: Achieving a consistent **15–30 FPS** on standard laptop hardware to maintain processing accuracy during high-speed movement.
• Precision / Recall: Prioritized as a safety-critical metric to ensure a zero-miss rate for obstacles within the collision path.
• F1 Score: Balances precision and recall to minimize "False Positives" (unnecessary braking alerts).
• mAP (mean average precision): Evaluates detection accuracy across Indian-specific classes like rickshaws, pedestrians, and animals.
• IoU (Intersection over Union): Measures the precision of the semantic segmentation mask for the drivable road area.
Also include benchmark comparison if available.

GitHub Submission Requirement

All teams must submit:

- A **GitHub repository link** containing the full project code
- A **well-documented README file** explaining:

Required README sections:

- Project overview
- Model architecture
- Dataset used
- Setup & installation instructions
- How to run the code
- Example outputs / results

Repositories should be **publicly accessible during evaluation**.

Declaration

We confirm that the proposed work is original and developed by the team AllianceNexus.

All team members agree to the rules and evaluation process.